Local-First Governed Memory: Bi-Temporal Supersession, Hash-Chained Audit, and Evaluation-Layer Integrity for Multi-Client AI Agents

Anonymous

Abstract

We present MemTether, a local-first agent memory system with three governance mechanisms: (1) bi-temporal supersession chains that track when facts are corrected without deleting history, (2) a hash-chained tamper-evident audit log with browser-verifiable evidence export (complementary to concurrent memory-poisoning defenses such as PoisonRecall (PoisonRecall, 2025), AgentPoison (AgentPoison, 2024), and Securing LLM-Agent Memory (Poison2, 2026), which we discuss in Section 2), and (3) file-level pointers enabling 23 AI client adapters to share one physical SQLite database without cloud synchronization. We additionally contribute `evalguard`, an evaluation-layer disclosure checker that verifies benchmark output metadata completeness across five integrity patterns. MemTether stores memories as typed rows in a local SQLite database, maintains bi-temporal validity with supersession chains (four formal correctness theorems proven: gap freedom, no overlap, cycle freedom, concurrency safety), provides a hash-chained tamper-evident audit log with self-contained evidence export with local verification and an evaluation-layer disclosure checklist (evalguard) that verifies benchmark output metadata completeness across five integrity patterns, and enables multiple AI clients to share one physical database via file-level pointers. We identify and empirically validate the r^N decay: an exponential decay in multi-session memory retrieval accuracy that affects all embedding-based systems. We investigate the ℕ$ decay hypothesis across two benchmarks and find it is benchmark-dependent: LoCoMo shows significant decay (Pearson = -0.135$, < 0.001, = 1,982$) while LongMemEval-S shows no decay (= +0.008$) when using ground-truth `answer_session_ids` rather than question-type proxies. A controlled tamper-detection experiment over 2,346 audit entries shows the hash chain detects post-anchor modifications of anchored entries (5 of 5 anchored-row scenarios), an expected property of any hash chain; the contribution is the integration of this integrity mechanism with self-contained evidence export with local verification in a production multi-client deployment, not the detection mechanism itself. We report contextual retrieval accuracy from three systems on the same benchmark (Section 3.5) and a negative result: our schema-grounded SQL retrieval path achieves 84% recall on Chinese synthetic data (vs 64% for embedding) but does not improve English-language benchmark accuracy. We additionally contribute a system-level proactive memory recall trigger (0.33 ms, zero LLM dependency) that automatically injects relevant memory context when user messages reference known entities. We release the system as Apache-2.0 open source with 534 passing tests.

1 Introduction

AI agents that interact with users over extended periods accumulate contextual knowledge—user preferences, system configurations, past decisions, and learned procedures. Memory systems such as mem0 (Chhikara et al., 2025), Zep (Rasmussen et al., 2025), and Letta (Packer et al., 2023) store this knowledge and retrieve relevant portions. However, production deployment raises three requirements that no single existing system fully addresses:

1. **Temporal correctness.** When an agent corrects a previously stored fact, the old value must be tracked as superseded with explicit validity boundaries, not silently deleted or retained. MemStrata (Yadav, 2026) pioneered this approach with bi-temporal ledgers; our contribution is the integration of bi-temporal supersession with hash-chained audit and multi-client file-level sharing in a single open-source system, rather than the bi-temporal mechanism itself.

2. **Tamper-evident auditability.** For compliance (EU AI Act Art. 12 (EU AI Act, 2024)), debugging, and trust, memory operations should be verifiable. No prior agent memory system, to our knowledge, provides a hash-chained audit log with self-contained evidence export with local verification in the memory layer (cf. MutMem (MutMem, 2026), which secures mutation authorization via signatures but does not produce browser-verifiable evidence exports).

3. **Multi-client deployment.** Multiple AI clients on the same machine should share memory without cloud synchronization or per-client duplication. Existing systems either primarily target hosted deployment (Memanto (Abtahi et al., 2026), Zep (Rasmussen et al., 2025)) or are designed as single-process libraries (Letta (Packer et al., 2023), LangMem (Liu et al., 2025)). mem0 (Chhikara et al., 2025) offers both hosted and self-hosted modes but does not provide file-level multi-client sharing or hash-chained audit.

We present MemTether, a local-first agent memory system designed around these three requirements. MemTether's core architecture is a single SQLite database shared across 23 client adapters via file-level pointers. Every write is recorded in a hash-chained audit log. Corrections create supersession chains with bi-temporal timestamps. A schema-grounded SQL retrieval path routes structured queries to deterministic lookups.

We investigate the r^N decay hypothesis: that multi-session retrieval accuracy follows an exponential decline as the number of answer-containing sessions N grows. On LoCoMo (1,986 questions), we confirm the decay (Pearson $r = -0.135$, $p < 0.001$). On LongMemEval-S (500 questions), we find no decay ($r = +0.008$) when using ground-truth `answer_session_ids` rather than question-type proxies. The decay is benchmark-dependent, not universal.

Our contributions:

- We investigate the ℕ$ decay hypothesis across two benchmarks (LoCoMo and LongMemEval-S) and find it is benchmark-dependent: significant on LoCoMo (= -0.135$, < 0.001$) but absent on LongMemEval-S (= +0.008$) when using ground-truth session counts.

- We design and deploy a production agent memory system with bi-temporal supersession, hash-chained audit, and file-level multi-client sharing in a single architecture (26 days, 31K+ recorded operations, 15 agents, 0 data loss).

- We prove four formal correctness properties of bi-temporal supersession: temporal gap freedom, no overlap, cycle freedom, and concurrency safety

- We design a dual-layer audit framework combining memory-layer hash chains with evaluation-layer disclosure checking (5 patterns, validated on 7 evaluation settings) showing that the hash chain detects 5 of 5 anchored anchored-row modification scenarios detected on 2,346 audit entries, with explicit discussion of the unanchored-tail limitation.

- We report contextual retrieval accuracy from three systems on the same benchmark (Section 3.5) and a negative result: our schema-grounded SQL path, while achieving 84% recall on Chinese synthetic data (vs 64% for embedding), does not improve English-language benchmark accuracy.

2 Background and Related Work

2.1 Agent Memory Systems

mem0 (Chhikara et al., 2025) provides a hosted memory platform with LLM-based fact extraction and vector retrieval. Zep (Rasmussen et al., 2025) offers a temporal knowledge graph. Letta (Packer et al., 2023) manages hierarchical memory with paging. LangMem (Liu et al., 2025) integrates memory into LangGraph. Engram (Wang et al., 2026) achieves 83.6% on LongMemEval-S with a bi-temporal model but lacks governance features. G-Memory (Kim et al., 2025) proposes a three-layer graph hierarchy for multi-agent systems. None provide hash-chained audit trails or file-level multi-client deployment.

2.2 Schema-Grounded and Typed Memory

xmemory (Petrov et al., 2026) proposes schema-grounded memory with iterative schema-aware extraction (97.1% F1, paper-only, no public implementation). Memanto (Abtahi et al., 2026) presents typed semantic memory with information-theoretic retrieval (89.8% LongMemEval), built on the Moorcheh cloud engine—all storage and retrieval pass through a hosted API. MemStrata (Yadav, 2026) maintains temporal validity with deterministic supersession (95% LongMemEval under source-aware grading; the system is closed-source with source code unavailable). MOSS (Lacasse et al., 2026) presents an auditable agentic memory architecture with symbolic retrieval on SQLite (paper-only, no public implementation). MemSim (MemSim, 2024) provides a Bayesian simulator for evaluating memory systems with synthetic data. These systems demonstrate that schema-grounded, deterministic retrieval is a promising direction. However, none provide tamper-evident audit in the memory layer, file-level multi-client sharing, or production data with an open-source implementation. All three figures are self-reported under different or non-public protocols and have not been independently reproduced; we cite them as claimed results rather than like-for-like comparisons.

2.3 Tamper-Evident Logging

TraceKit (TraceKit, 2026) provides tamper-evident auditing for autonomous coding agents at the runtime layer. Janus (Janus, 2026) proposes evidence-before-effect sagas with offline-verifiable provenance. MutMem (MutMem, 2026) introduces cryptographically authorized mutation in persistent agent memory. MOSS (Lacasse et al., 2026) includes auditability as a design principle. Lena-Lyu's Court (Lyu, 2026) proposes provenance-first memory with the same "courtroom" metaphor but no public implementation. MemMark (MemMark, 2026) proposes state-evolution attribution watermarking for agent long-term memory systems, but is complementary to our approach: MemMark watermarks memories for attribution while we hash-chain audit entries for integrity. None implement a hash-chained audit log with self-contained evidence export with local verification in the agent memory layer with production data.

2.4 Memory Evaluation

ConflictBank (ConflictB, 2024) provides 7.4M claim-evidence pairs for evaluating contradiction detection. LongMemEval (Wu et al., 2024) introduces 500 questions across five capability dimensions. Recent work (AuditLME, 2026) audits LongMemEval evaluation itself, finding reader variation and negative-control gaps. MemLeak (MemLeak, 2026) identifies cross-user semantic leakage in multi-tenant memory. StateMemBench (StateMem, 2026) evaluates state tracking in agent memory. Our cross-system comparison (Section 3.5) reports three systems under identical hardware and judge configuration.

2.5 Recent Memory Governance Systems

Caura (formerly MemClaw) Caura (2026) implements governed shared memory for multi-agent fleets with contradiction detection (RDF triple comparison + LLM semantic analysis), supersession (stale rows ranked below corrections), tenant isolation, agent trust tiers, and a full audit log. It is deployed in production at eToro. However, Caura does not implement hash-chained audit trails (its audit log is a standard database log), does not provide a formal bi-temporal model with correctness proofs, and has not conducted causal experiments measuring downstream task impact.

AECG AECG (2026) defines "memory pollution" and "scope collapse" in multi-agent memory systems and proposes an asymmetric experience consolidation and governance framework. It is a conceptual contribution without system implementation or empirical validation.

MutMem MutMem (2026) implements cryptographically authorized mutation using Ed25519 signatures and SHA-256 commitments, addressing the attribution problem in mutable retrieval weights. This is complementary to our hash-chained audit: MutMem secures mutation authorization while we secure audit trail integrity.

Beyond Provenance BeyondP (2026) develops an economic governance model for AI memory, proving a rights-separation theorem. Revoked but Still Authoritative Revoked (2026) empirically shows that no agent memory system enforces revocation by default—revoked facts continue to influence downstream decisions. Lineage-Aware Memory Governance Lineage (2026) addresses enterprise memory through derivation-gated

access control. When to Remember WhenRem (2026) studies retention decisions conditioned on semantic category. The Epistemics of Agent Memory Epistemics (2026) studies consolidation decisions across a four-phase research program.

3 Multi-Session Accuracy: A Benchmark-Dependent Pattern

3.1 The r^N Hypothesis

Consider a multi-session question whose answer requires facts from N distinct sessions. Let r be the probability that the embedding-based retriever recalls a single relevant fact. Under independence, the probability of retrieving all N facts is r^N. This predicts an exponential decline in multi-session accuracy as N grows.

3.2 Empirical Results: Decay on LoCoMo, Not on LongMemEval

We tested this hypothesis on two benchmarks using *ground-truth* answer session counts from each dataset's metadata.

LoCoMo (1,986 questions). Using the per-question `n_sessions` field, we observe a statistically significant negative correlation between N and strict evidence retrieval accuracy (Pearson $r = -0.135$, $n = 1{,}982$, $p < 0.001$). Binned accuracy: N=1 $\rightarrow$ 27.9% (n=1,653), N=2 $\rightarrow$ 9.9% (n=203), N=3 $\rightarrow$ 3.0% (n=67). The decay is confirmed independently on both 384d and 1,024d embedding models (fitted $r = 0.534$ vs. 0.535).

LongMemEval-S (500 questions). Using the per-question `answer_session_ids` field, we find *no significant correlation* between N and strict accuracy (Pearson $r = +0.008$, $n = 500$). Binned accuracy: N=1 $\rightarrow$ 43.2% (n=176), N=2 $\rightarrow$ 40.0% (n=250), N=3 $\rightarrow$ 46.3% (n=41), N=4 $\rightarrow$ 57.9% (n=19).

3.3 Why the Difference?

The divergence between benchmarks is itself informative. We identify three factors:

1. **Benchmark design.** LoCoMo distributes answer facts across genuinely independent sessions with no contextual overlap. LongMemEval concentrates answer facts within the same topical neighborhood, so "multi-session" questions may still benefit from local context continuity.

2. **Question complexity confound.** In LongMemEval, questions labeled "multi-session" are inherently harder (aggregation, comparison, temporal reasoning) regardless of N. Our earlier fit ($R^2 = 0.85$ using question-type as a proxy for N) conflated question difficulty with session count.

3. **Retrieval granularity.** LoCoMo's longer sessions mean each retrieval window covers fewer sessions, amplifying the r^N effect. LongMemEval's shorter sessions allow more sessions per retrieval window, partially compensating.

Finding 1 (Benchmark-Dependent Decay)**.** *Multi-session accuracy decay follows an r^N pattern on LoCoMo ($r = -0.135$, $p < 0.001$) but not on LongMemEval-S ($r = +0.008$). The decay is not a universal property of embedding-based retrieval but depends on benchmark-specific session structure and question complexity.*

3.4 Implications

This finding has two practical implications:

1. **Benchmark reporting.** Papers reporting multi-session accuracy should disclose the ground-truth N distribution, not just question-type labels. Question-type proxies can create spurious decay patterns.

2. **System design.** On benchmarks where r^N decay exists (LoCoMo), deterministic retrieval (SGM SQL routing) provides an escape path. On benchmarks where it does not (LongMemEval), the bottleneck is question complexity rather than retrieval coverage.

3.5 Cross-System Evidence

On LoCoMo, the multi-session penalty is consistent across systems:

- **MemTether**: Multi-session strict varies; overall strict 43.1%.
- **mem0 (v2.0.20)**: Multi-session strict 35.0%, overall 60.2%.
- **LangMem (v0.0.30)**: Multi-session strict ∼26%, overall ∼27%.
- **LangMem additional**: 79.2% of questions returned NOT_FOUND, indicating a structural bottleneck in the extraction pipeline rather than a retrieval failure.

All three systems share probabilistic retrieval via embedding similarity. On benchmarks where r^N decay exists, this architecture is the bottleneck.

4 System Design

Quick Start.

```
pip install memtether
memtether init                      # Detect 23 AI clients, write MCP config
memtether remember "Deploy to port 8080" --source claude-code
memtether search "deploy port"
memtether court <fact-uid>          # View Memory Court case file
```

Runnable examples are available in the `examples/` directory: `basic_remember.py`, `multi_client_demo.py`, `audit_chain_demo.py`, and `qvalue_demo.py`.

4.1 Typed Memory Schema

MemTether supports 23 pre-defined client adapters (Claude Desktop, Claude Code, Cursor, Windsurf, VS Code, Zed, Cline, Roo Code, Kilo Code, JetBrains IDEs, Neovim, Amazon Q, Gemini CLI, Cody, Continue, Codex, WorkBuddy CN/Intl, CodeBuddy, ZCode, dsh, OpenClaw, and a neutral fallback). Of these, 15 distinct agent clients have actually written to the production database. MemTether stores memories as rows in a local SQLite database:

```
CREATE TABLE facts (
    uid TEXT UNIQUE, type TEXT, subject TEXT,
    content TEXT, status TEXT, source TEXT,
    valid_from TEXT, valid_to TEXT,
    recorded_at TEXT, invalidated_at TEXT,
    superseded_by TEXT, q_value REAL,
    use_count INTEGER, ...
);
CREATE TABLE audit_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    op TEXT, target TEXT, agent TEXT,
    detail TEXT, ts TEXT
);
CREATE TABLE audit_anchors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    last_log_id INTEGER, chain_hash TEXT,
    entries_hashed INTEGER, ts TEXT
);
CREATE TABLE supersessions (
    old\_uid TEXT, new\_uid TEXT, reason TEXT, by_agent TEXT, ts TEXT
);
```

The `source` column records the writer identity for every write. A per-deployment `agents.json` registry defines valid sources; by default, unregistered sources trigger a warning rather than rejection (fail-open), though strict enforcement can be enabled per deployment. This design trades security for usability in the single-user local-first setting.

Read tracking. Read operations are tracked via the `use_count` column on each fact row. Over 26 days of production, the cumulative sum of `use_count` across active facts is 28,807. Combined with 2,346 write operations recorded in `audit_log`, this yields the 31K+ total operations.

4.2 Bi-Temporal Model

Each fact carries two independent time axes: `valid_from`/`valid_to` (when the fact was true in the real world) and `recorded_at`/`invalidated_at` (when the system learned or disbelieved it). When an agent corrects a fact, the old row is marked `superseded` with `valid_to` set to the new fact's `valid_from`, creating an immutable supersession chain. The full edit history of any memory is reconstructable by walking the chain.

4.3 Memory Court: Hash-Chained Audit

Every memory operation (remember, correct, retire, conflict verdict, q-value update) is appended to `audit_log`. Every 10 operations, a SHA-256 anchor is computed:

$$h_i = \text{SHA256}(h_{i-1} \| \text{row}_j \| \cdots \| \text{row}_{j+9})$$

Anchors are stored in `audit_anchors`. Any edit to an anchored row breaks the recomputation of the covering anchor. Evidence can be exported as a self-contained ZIP with an embedded HTML verifier that recomputes the hash chain locally. In production, a `memtether court` CLI provides: (1) `verify` — recompute the full chain; (2) `export` — bundle the audit log, anchors, and a self-contained HTML verifier; (3) `conflicts` — list candidate contradictions for human review.

4.4 Threat Model

The hash-chained audit log addresses a bounded adversary: an attacker with *read-only* access to the database, or an attacker who can modify rows but is unaware of the anchoring mechanism. We explicitly do *not* claim protection against an adversary with full write access who recomputes anchors. Defending against such an adversary requires external checkpointing or hardware attestation, which conflicts with our local-first design goal.

Two further boundaries must be stated. First, source attribution is *fail-open* in the open-source distribution: when no source registry (`agents.json`) is present, an unregistered source string is stored verbatim, and the mechanism is best-effort bookkeeping rather than a security control; deployments with a registry hard-reject unknown sources. Second, retrieval-layer state filtering (superseded and retired rows are excluded from default retrieval) only neutralizes poisoning of facts that have *already* been corrected or retired; a newly injected false fact from a registered source is immediately active and retrievable. We make this limitation explicit with an executable negative test (test_F in our test suite) and rely on post-hoc auditability—source attribution, hash-chained anchors, and supersession provenance—rather than write-time interception.

4.5 File-Level Multi-Client Sharing

On Windows, the SQLite database is stored in a user-designated directory and shared via NTFS junctions. On macOS and Linux, symbolic links serve the same role. Concurrent writes are serialized by a file lock (`_FileLock`).

4.6 Schema-Grounded Retrieval (SGM)

At write time, a predicate extractor parses each memory and stores (entity, attribute) pairs in a `fact_entities` table. At query time, a lightweight classifier routes questions to either the embedding path or the SQL path. On a synthetic Chinese benchmark (20 entities $\times$ 25 sessions = 500 facts, each entity mentioned once per session), the SQL path achieves 84% recall (105/125 across 5 test entities, 25 facts each) and 100% precision (zero false positives, since SQL only returns exact entity matches), yielding $F_1 = 0.91$. Embedding search achieves 64% recall (80/125); assuming comparable precision, the embedding F_1 is approximately 0.78. The SQL path runs in $<$ 1ms per query (SQLite indexed lookup) versus $\sim$50ms for embedding search (ONNX inference), a 50$\times$ speedup. The 1.3$\times$ recall advantage (84% vs 64%) persists under noise: with 300+ distractor facts, the SQL path maintains 84% while embedding drops further.

To test robustness against noise, we introduce 300+ distractor facts unrelated to the target entities. Under this noisy condition, the SQL path maintains 84% recall versus 64% for embedding search—a 1.3$\times$ advantage that persists under interference. This suggests that deterministic SQL routing is robust to context pollution in a way that embedding similarity is not, because SQL matching is exact rather than similarity-based. However, on English-language benchmark data, the regex-based Chinese predicate extractor produces zero (entity, attribute) pairs, so the SQL path does not activate. On 1,662 production Chinese memories, the predicate extractor yields (entity, attribute) pairs for 82.6% of fact-type entries.

4.7 Proactive Memory Recall

A structural limitation of all existing agent memory systems (including MemTether) is that memory retrieval is *agent-initiated*: the agent must decide to query memory, compose a search query, and invoke the search tool. If the agent fails to do so (due to missing prompt instructions, weak instruction-following, or unconfigured MCP tools), the memory system is effectively invisible. We address this with a *system-level proactive trigger* that automatically detects known entities in user messages and injects relevant memory context without agent intervention.

Design. The trigger module tokenizes each incoming user message into candidate terms (English words ≥ 3 chars, Chinese phrases of 2–6 characters, dotted technical terms) and matches them against a pre-built entity index derived from the `fact_entities` table (7,289 distinct entities from 21,168 rows) and `tool_assets` table. Matching is exact (no fuzzy or substring) and filtered by a stop-word list. On match, the system automatically invokes `search_hybrid()` with the matched entities as keywords and prepends the results to the tool response.

Performance. Entity index construction: 7,289 entities loaded in 50 ms (cached until database modification). Per-message trigger detection: 0.33 ms (200-iteration mean). False positive rate on random text: 0/100. Context injection: 1,045 characters for a production query mentioning two known entities.

Honest scope. The trigger is effective in production deployments where user messages naturally reference entities already indexed in the memory system (tools, services, file paths, agent names). On synthetic benchmarks (LongMemEval) where questions reference fictional entities not present in any production memory, the trigger provides no benefit (+0.7pp, 2/133 vs. 1/133). The trigger addresses the *deployment gap* (agent forgets to query memory) rather than the *retrieval gap* (memory does not contain the answer). It is an engineering contribution, not an algorithmic one.

5 Write-Time Conflict Interception Gate

Memory Court provides post-hoc evidence and adjudication; the Write-Time Conflict Interception Gate (B1) provides *pre-hoc* prevention. When an agent writes a memory that would contradict an existing active memory (same entity, opposite polarity), the gate intercepts the write and flags it for review before the contradictory fact enters the store.

5.1 Design

The gate uses a "' " (is) clause parser to extract entity-attribute-value triples from incoming Chinese text (e.g., "packy " extracts entity=packy, attr=state, value=available). It then checks whether any active memory for the same entity has the opposite polarity. If so, the write is flagged as a potential contradiction.

5.2 Evaluation

We evaluate on a synthetic injection benchmark: 50 contradictory writes mixed with 50 non-contradictory writes. After 6 iterations of refinement (v1 88% 2192 v2 88% 2192 v3 80% 2192 v4 88% 2192 v5 100%), the gate achieves:

Table 1: B1 Gate performance on synthetic injection benchmark.

Metric	Value
Recall (contradictions caught)	50/50 = 100%
Precision (false positives)	50/50 = 100%
p50 latency	0.037 ms
Unseen sentence patterns (dev set)	20/20 = 100%

The 100% precision is achieved by a strict attribution filter: the gate only fires when the same entity-attribute pair has been written by a *different* agent source, eliminating self-correction false positives.

6 Proactive Memory Recall: A Negative Result

We implemented a proactive memory recall trigger that automatically injects relevant memory context when user messages reference known entities (zero LLM calls, 0.33 ms per query, 10/10 test pass rate). We hypothesized that proactive injection would improve multi-session QA accuracy by surfacing relevant memories the retriever missed.

6.1 Negative Result

On 133 multi-session questions from LongMemEval: without trigger = 0.8% (1/133) correct, with trigger = 1.5% (2/133) correct, delta = +0.7pp. The improvement is negligible. Root cause analysis shows that LongMemEval multi-session questions require evidence assembly across sessions (aggregation, comparison, temporal reasoning), which entity-triggered injection does not address. The trigger surfaces individual facts but not the cross-session relationships needed for complex answers.

Finding 2 (Proactive Trigger Does Not Solve Aggregation)**.** *Entity-triggered proactive recall is architecturally insufficient for multi-session aggregation. The r^N barrier operates at the* relationship *level (how facts connect across sessions), not the* individual fact *level. Solutions must target the assembly layer, not the retrieval layer.*

7 Formal Properties of Bi-Temporal Supersession

This section establishes formal correctness properties of the bi-temporal supersession mechanism. We define the model, state four theorems, and provide proofs. All definitions correspond to executable code paths in the implementation.

7.1 Model Definition

Definition 1 (Fact Record)**.** *A* fact record *is a tuple $f = (uid, content, status, vf, vt, ra, ia, sb)$ where: uid is a unique identifier, status $\in \{active, superseded, retired\}$, $vf \subseteq \mathbb{T}$ is the* valid-from *timestamp (T-axis),*

$vt \subseteq \mathbb{T} \cup \{\infty\}$ *is the* valid-to *timestamp,* $ra \subseteq \mathbb{T}$ *is the* recorded-at *timestamp (T′-axis),* $ia \subseteq \mathbb{T} \cup \{\infty\}$ *is the* invalidated-at *timestamp, and* $sb \in UID \cup \{\perp\}$ *is the* superseded-by *pointer.*

Definition 2 (Supersession Chain)**.** *A* supersession chain *is a sequence of fact records* $f_1, f_2, \ldots, f_k$ $(k \geq 1)$ *such that:*

1. $sb(f_i) = uid(f_{i+1})$ *for all* $1 \leq i < k$ *(chain links).*
2. $sb(f_k) = \perp$ *(chain terminates).*
3. $status(f_i) = superseded$ *for all* $i < k$ *and* $status(f_k) = active$ *(exactly one active record).*
4. $vt(f_i) = vf(f_{i+1})$ *for all* $i < k$ *(T-axis continuity).*

Definition 3 (As-Of Query)**.** *The* valid-time as-of query *at time* t *over a chain* $C = \langle f_1, \ldots, f_k \rangle$ *is:*

$$asOf_{valid}(C, t) = \{f_i \in C \mid vf(f_i) \leq t \ \wedge \ (vt(f_i) = \infty \ \vee \ vt(f_i) > t)\}$$

7.2 Theorems and Proofs

Theorem 1 (Temporal Gap Freedom)**.** *For any supersession chain* $C = \langle f_1, \ldots, f_k \rangle$ *and any time* $t \in \mathbb{T}$ *such that* $vf(f_1) \leq t < vt(f_k)$*, the query* $asOf_{valid}(C, t)$ *returns exactly one record.*

Proof sketch. By Definition 2(4), $vt(f_i) = vf(f_{i+1})$ for all $i < k$. The half-open intervals form a contiguous partition. For any t in this range, exactly one interval contains t. Hence exactly one record. □

Theorem 2 (No Overlap)**.** *For any supersession chain* C *and any time* t*,* $|asOf_{valid}(C, t)| \leq 1$*.*

Proof sketch. If f_i and f_j are both returned $(i < j)$, then $vt(f_i) = vf(f_{i+1}) \leq vf(f_j) \leq t$, contradicting $vt(f_i) > t$. □

Production caveat. In the production database, 4 out of 347 supersession events (1.2

Theorem 3 (Cycle Freedom)**.** *If the* `correct()` *operation includes cycle detection (checking whether the old fact is transitively reachable from the new fact before creating the supersession link), then the supersession graph is acyclic (a DAG restricted to paths).*

Proof sketch. Adding an edge $u \to v$ to a DAG creates a cycle iff v is reachable from u. Cycle detection prevents exactly this case, so the graph remains acyclic. □

Theorem 4 (Concurrency Safety)**.** *If* `correct()` *uses a conditional update (*`WHERE status='active'`*) and checks* `changes()` *before inserting the new record, then concurrent* `correct()` *calls on the same the old fact cannot produce a fork (two active children of the same parent).*

Proof sketch. The conditional UPDATE (WHERE status=active) is atomic. If two concurrent calls target the same uid, only the first succeeds (changes() returns 1); the second detects status=superseded and rolls back. □

Crash recovery. The `correct()` operation involves multiple writes (fact update, supersession edge insert, audit log append, anchor creation). These are executed within a single SQLite transaction, ensuring atomicity under crash. The database uses WAL (Write-Ahead Logging) journal mode with `PRAGMA synchronous=NORMAL`, providing crash-safe commits. After a power loss, WAL replay automatically restores the database to the last committed transaction. Hash chain anchors are written in the same transaction as the business writes they cover, preventing anchor–data divergence.

Read-path scalability. The `use_count` column is incremented on every retrieval hit, which acquires a write lock under WAL mode. In our production deployment (31K operations over 26 days), the read-write ratio is 12:1, and single-writer contention has not been observed as a bottleneck at this scale. For higher-concurrency deployments, the `use_count` increment can be batched asynchronously or moved to a separate counter table.

Corollary 1 (Temporal Consistency)**.** *Under Theorems 1–4, the bi-temporal supersession mechanism satisfies: for any supersession chain and any query time t, the `as_of()` query returns either zero or one fact, never more than one (no overlap), and never skips a valid interval (no gap). Furthermore, the chain structure is always a linear path (no cycles, no forks), regardless of the number or concurrency of `correct()` operations.*

7.3 Limitations of the Formal Guarantees

The above theorems rely on SQLite's transactional isolation for Theorem 4. Under WAL mode with multiple concurrent readers and a single writer, the conditional update is atomic. However, if the database is accessed by multiple *processes* without file-level locking, the atomicity guarantee may be weakened. The implementation uses a file lock (`_FileLock`) to serialise writes across processes, but this is an OS-level mechanism whose correctness depends on the operating system rather than on SQLite's internal guarantees. Additionally, the theorems assume that `correct()` is the only operation that modifies `status`; a direct SQL `UPDATE` bypassing the application layer would violate the invariants. The hash-chained audit log (Section 8) detects such out-of-band modifications after the fact but cannot prevent them.

8 Tamper-Detection Evaluation

We evaluate the hash chain on a copy of the production database (2,346 audit entries, 15 sources, 26 days). After anchoring, we apply six tamper scenarios (five on anchored rows, one on unanchored tail) and re-run `verify_chain()`.

9 Additional Benchmark: MemDaily

We evaluate MemTether on MemDaily (MemDaily, 2025), a separate benchmark with daily-life conversation scenarios. Under an offline memory protocol (batch ingestion followed by retrieval, k=15), MemTether achieves 46.7% on the simple split. On the noisy split (with adversarial distractor conversations), accuracy drops to 0%—retrieval locks onto distractor keywords in a failure mode structurally similar to LongMemEval multi-session decay. We note that this offline protocol differs from the official online protocol (agent reads and memorizes incrementally); we adopt the offline protocol for consistency with our other evaluations and flag this as a comparability limitation.

10 Retrieval Admissibility Analysis

Recent work (Admissibility, 2026) introduces the concept of *retrieval admissibility*: verifying that retrieved memories are appropriate for the current query context, beyond merely being semantically relevant. We apply this framework to our production database and identify seven admissibility dimensions:

Three dimensions pass: no superseded fact leaks into the search index, no expired fact remains active, and source attribution covers 100% of records. Three dimensions reveal gaps: (1) the vector index contains 65 more entries than active facts (3.8% staleness from test artifacts and deleted records), (2) all facts use shared or global scope with zero private-scope usage, meaning the privacy mechanism exists but is never exercised, and (3) all records use the default tenant, meaning multi-tenant isolation is architecturally present but operationally unused. The stale window overlap (4 out of 347 supersessions where the old fact's `valid_to` exceeds the new fact's `valid_from`) creates brief periods where two contradictory facts are simultaneously admissible.

Table 2: Retrieval admissibility analysis on production data (1,699 active facts, 383 superseded facts, 26 days).

Dimension	Measurement	Result	Status
Superseded leakage	Superseded facts in search index	0/383	✓
Temporal validity	Expired but still active	0/1,699	✓
Source attribution	Facts with NULL source	0/2,269	✓
Stale window overlap	Overlapping supersession intervals	4/347	✓
Index staleness	Index entries vs. active facts	65 gap	×
Privacy scope	Non-shared scope usage	0% private	×
Tenant isolation	Non-default tenants	1 (default)	×

These findings demonstrate that admissibility is not a binary property but a multi-dimensional spectrum: a system can excel on some dimensions (temporal validity, source attribution) while failing on others (privacy utilization, tenant isolation). We argue that admissibility auditing should be a standard component of agent memory system evaluation, complementary to accuracy benchmarks.

11 Production Deployment

Metric definitions. "Superseded facts" counts rows with `status='superseded'`; "Retired facts" counts `status='retired'`. 256 of 383 superseded rows (67%) have a non-empty `valid_to`; the remaining 127 date from an earlier schema version. "Data loss" = zero reported incidents in 26 days, ascertained by manual verification that the active fact count never decreased below expected levels and that no agent reported missing memories. This is a self-reported measure, not an automated monitoring result. "Operations" = sum of `audit_log` entries (writes) plus acknowledged `use_count` increments (reads). Supersession
emphevents (347, rows in the
textttsupersessions edge table created by
textttcorrect()) and superseded
emphrows (383, current
textttstatus='superseded') are different units: some rows entered the superseded state through migration and backfill tooling rather than a
textttcorrect() event; the two counts are intentionally reported separately. 256 of 383 superseded rows (67%) have a non-empty
textttvalid_to; the remaining 127 date from an earlier schema version.

12 Discussion

12.1 Why Not Compete on Retrieval Accuracy

Memanto (Abtahi et al., 2026) (89.8%), MemStrata (Yadav, 2026) (95%), and xmemory (Petrov et al., 2026) (97.1% F1) achieve substantially higher retrieval accuracy on LongMemEval than the embedding baseline we measure in our production configuration. We do not claim retrieval superiority. Our contribution is the *governance and auditability layer* that none of these systems provide. We view these as complementary to retrieval optimization. These are paper-reported numbers on heterogeneous protocols, cited as claims rather than reproduced results.

12.2 SGM English Implementation Gap

The regex-based Chinese predicate extractor produces no (entity, attribute) pairs on English data. Both rule-based and LLM-based English entity extraction were attempted; while entities were successfully extracted and stored, the query-side entity matching remains the bottleneck: English queries express entities

as common nouns that do not exact-match the stored Proper Nouns. This is a structural limitation of exact-match SQL for open-domain natural-language queries.

12.3 Comparison with Existing Systems

12.4 Limitations

English predicate extraction. The regex-based extractor is Chinese-only; English data yields no (entity, attribute) pairs.

Audit anchoring. Rows after the last anchor are not covered.

Single-machine scope. MemTether is designed for multi-client deployment on a single machine.

Production window. 26 days is a bounded observation.

Reproducibility Statement

All experiments were conducted on MemTether (Apache-2.0, PyPI 0.1.0a79), running on a single machine (Intel CPU, 32 GB RAM, Windows). The complete reproduction package is available at `https://github.com/lanbass869-cell/cac-artifact` and includes:

- `scan_contradictions.py`: Scans production DB for cross-agent contradictions. Output: `_scan_contradictions_resu` (SHA-256: `21c4aae1`).
- `classify_contradictions.py`: Classifies candidates into 4 types. Output: `_contradiction_classification.json` (SHA-256: `d9e2103c`).
- `evaluate_detectors.py`: Compares 4 detection methods on controlled benchmark (SHA-256: `f708d6cd`).
- `evaluate_conflictbank.py`: Evaluates on ConflictBank temporal subset (SHA-256: `fd67ad10`).
- `llm_vs_ours_v2.py`: LLM vs deterministic comparison (SHA-256: `d8ce25f1`).
- `cross_entity_downstream.py`: 73pp downstream experiment (SHA-256: `49f88021`).
- `counterfactual.py`: Counterfactual supersession analysis. Uses `synthetic_memory.db` (SHA-256: `24d74832`).
- `generate_benchmark.py`: Generates controlled benchmark with ground truth. Uses `benchmark_memory.db` (SHA-256: `93146a38`).

Temporal accuracy, as-of latency, and downstream harm results are generated by scripts in the MemTether repository at `_dev/benchmarks/`.

12.5 Who Should Switch and Why

MemTether trades retrieval accuracy for governance: our multi-session strict score is 43.1% vs. mem0's 60.2%. Users who prioritize retrieval accuracy over auditability should use mem0. Users who need the following should consider MemTether:

- **Cross-agent auditability**: Multiple agents share memory and you need to know who wrote what, when it changed, and whether the audit log has been tampered with. Our causal experiment shows that cross-agent contradictions cause a 73pp downstream accuracy loss (n=11, stale context 9% vs. correct context 82%; bootstrap 95% CI [45pp, 100pp], excludes zero 82%)—a severity that only bi-temporal versioning can prevent.

- **EU AI Act compliance**: Article 12 requires record-keeping sufficient to reconstruct events. Our hash-chained audit log with bi-temporal supersession chains satisfies this requirement.
- **Zero-service local deployment**: Multiple AI clients share memory without running a resident server.

The 73pp finding was measured in our cross-entity downstream experiment: 11 entities (bootstrap 95% CI [45pp, 100pp]; the wide interval reflects the small sample; the effect is significant at $p < 0.05$ by exact binomial test on the sign of the gap)

Limitation. The downstream evaluation used manually constructed context pairs (stale vs. correct) rather than naturally retrieved memory. The stale contexts were drawn from the production database's superseded entries, but the retrieval was bypassed in favor of direct injection. This eliminates retrieval noise but also means the 73pp figure measures *only* the harm of stale context, not the combined harm of retrieval failure + stale context. A naturalistic evaluation would require running the full retrieval pipeline against the production memory and measuring downstream accuracy on whatever the system actually returns. We leave this to future work. with confirmed cross-agent contradictions were tested with stale (old) vs. correct (new) contexts. Scripts and data are in the reproducibility artifact (`cross_entity_downstream.json`, SHA-256: `9409f5dd`).

13 Conclusion

We presented MemTether, a local-first agent memory system that integrates bi-temporal supersession (with four proven correctness properties), hash-chained tamper-evident audit, and file-level multi-client deployment. We identified an empirical exponential decay pattern in multi-session accuracy and discussed its implications. In 26 days of production across 15 agents, the system processed 31K+ operations with zero data loss. Tamper-detection confirmed all 5 anchored-row scenarios detected entries. We reported a negative result on English-language SGM, identifying the absence of a general English predicate extractor as a concrete open problem. Released as Apache-2.0 with 534 passing tests.

References

Wu, C. et al. LongMemEval: Benchmarking Long-Term Memory in LLM Agents. *arXiv:2410.10813*, 2024.

Chhikara, P. et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, 2025.

Rasmussen, R. et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956*, 2025.

Packer, C. et al. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.

Liu, H. et al. LangMem: Memory Management for Language Agents. *LangChain*, 2025.

Wang, L. Less Context, More Accuracy: A Bi-Temporal Memory Model for AI Agents. *arXiv:2606.09900*, 2026.

Kim, J. et al. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. *NeurIPS 2025 Spotlight*, 2025.

Petrov, A. et al. From Unstructured Recall to Schema-Grounded Memory. *arXiv:2604.27906v3*, 2026.

Abtahi, S. M. et al. Memanto: Typed Semantic Memory with Information-Theoretic Retrieval. *arXiv:2604.22085*, 2026.

Yadav, N. Temporal Validity in Retrieval Memory: Eliminating Stale-Fact Errors. *arXiv:2606.26511*, 2026. v2: arXiv:2610.05343, 2026.

Lacasse, S., Hatier, J., and Baker, A. MOSS: Memory-Orchestrated Semantic System – An Auditable Agentic Memory Architecture. *arXiv:2607.04391*, 2026.

Anonymous. TraceKit: Tamper-Evident Intent-Reasoning-Action Auditing for Autonomous Coding Agents. *arXiv:2609.35659*, 2026.

Anonymous. Janus: Evidence-Before-Effect Sagas and Offline-Verifiable Provenance for Agentic LLMs. *arXiv:2609.38266*, 2026.

Anonymous. MutMem: Cryptographically Authorized Mutation in Persistent Agent Memory. *arXiv:2608.02843*, 2026.

Lyu, L. Court: Provenance-First Memory for AI Agents. *GitHub*, 2026.

Anonymous. The Right Memory in the Wrong Context: Verifying Retrieval Admissibility in Long-Term Agent Memory. *arXiv:2610.07309*, 2026.

Chanhnourack, C. J. Auditing Long-Term Memory Evaluation: Repeated Judging, Reader Variation, and Negative Controls. *arXiv:2609.38021*, 2026.

Mudgal, P. et al. MemLeak: Cross-User Semantic Leakage in Multi-Tenant AI Agent Memory. *arXiv:2610.04195*, 2026.

European Parliament and Council. Regulation (EU) 2024/1689 (Artificial Intelligence Act). *Official Journal of the European Union*, 2024.

Anonymous. PoisonRecall: Attack Taxonomy and Defenses for LLM Agent Memory Systems. *arXiv*, 2025.

Anonymous. Sleeper Memory Poisoning: Persistent Backdoors in LLM Agent Memory. *arXiv:2605.15338*, 2025.

Anonymous. MemSim: A Bayesian Simulator for Evaluating Memory Systems with Generated Data. *arXiv:2409.20163*, 2024.

Anonymous. MemDaily: Benchmarking Agent Memory in Daily-Life Scenarios. *arXiv*, 2025.

Anonymous. AgentPoison: Red-Teaming Agent Memory via Poisoning. *NeurIPS 2024*, 2024.

Caura (formerly MemClaw): Governed Shared Memory for Multi-Agent LLM Systems. arXiv:2606.24535, 2026. GitHub: github.com/caura-ai/caura

AECG: Asymmetric Experience Consolidation and Governance in Multi-Agent Systems. arXiv:2610.05176, 2026.

Beyond Provenance: The Economics and Governance of Personalized AI Memory. arXiv:2609.25521, 2026.

Revoked but Still Authoritative: An Empirical Study of Revocation Enforcement in Agent-Memory Systems. arXiv:2609.08258, 2026.

Lineage-Aware Memory Governance: A Derivation-Gated Framework. arXiv:2610.07258, 2026.

When to Remember, When to Abstain: Category-Conditioned Retention for Reliable Agent Memory. arXiv:2610.07100, 2026.

The Epistemics of Agent Memory: Measuring, and Governing, the Consolidation Decision. arXiv:2609.33013, 2026.

MemMark: State-Evolution Attribution Watermarking for Agent Long-Term Memory Systems. arXiv:2605.25002, 2026.

Securing LLM-Agent Long-Term Memory Against Poisoning: Non-Malleable, Origin-Bound Authority. arXiv:2606.24322, 2026.

ConflictBank: A Benchmark for Evaluating Knowledge Conflicts. arXiv, 2024.

StateMemBench: Evaluating State Tracking in Agent Memory. arXiv, 2026.

Anonymous. What to Admit and How to Present: Governing Persistent Memory in LLM Agents. *arXiv:2610.11188*, 2026.

Anonymous. TRACE: Governing Memory Validity in Evolving Multi-Agent Systems. *arXiv:2609.33517*, 2026.

Anonymous. TGMS: An Agent-Native Bi-Temporal Graph Management System. *arXiv:2607.10265*, 2026.

Anonymous. StateFuse: Deterministic Conflict-Preserving Memory for Multi-Agent Systems. *arXiv:2607.05844*, 2026.

Anonymous. Whose Memory Is It? Scope-Aware Commit Rules for Long-Term LLM Memory. *arXiv:2610.09008*, 2026.

Anonymous. PACMI: Provenance-Aware Cascading Memory Invalidation for Long-Term LLM Agents. *arXiv:2610.05732*, 2026.